Version 1.17
10/28/2026

NMC2v3 Bring-Up Guide: Cabling, Programming & Loading Linux or RTEMS

WARNING: Wear an ESD wrist strap whenever you handle the NMC board. Electrostatic discharge can permanently damage the NMC board.

Overview

Going from a bare NMC2v3 to a running operating system takes five steps. Steps 1 to 4 set up and program the board, once per board. Step 5 loads Linux or RTEMS onto the eMMC, and you can repeat it any time to switch.

1. Install the software.
2. Download the release files.
3. Cable the board and open the serial terminals.
4. Program the FPGA design and the HSS bootloader (`nmc-fpga.job`).
5. Load an operating system: Linux with Buildroot (5A), Linux with Yocto (5B), or RTEMS (5C).

Steps 1 to 5 use the pre-built release files. The appendices cover the optional parts: building Linux (A, B) and RTEMS (C) yourself, changing the FPGA design (D) and bare-metal programs (E). The same steps are in the READMEs of the `nmc-bsp` repository (https://github.com/nextage-llc/nmc-bsp); if this guide and a README ever differ, the README is correct.

Have on hand:

- The NMC2v3 board and a 5 V power supply, current-limited to 2 A
- A Microchip FlashPro5 or FlashPro6 programmer
- A Windows PC
- The cables listed in Step 3 (for Linux, also an Ethernet cable)

Step 1: Install the software

Programming the board and loading the pre-built images only needs free tools. WSL2 is needed only to build images yourself.

FlashPro Express (to program the board)

1. Install Libero SoC 2025.2 from Microchip's FPGA Software Downloads page; it includes FlashPro Express. If you only program boards, the standalone Microchip Program and Debug Tools are enough.
2. Keep the default settings and pick any install directory.
3. FlashPro Express runs without a license. Libero needs one only to change the FPGA design (Appendix D).

Tera Term and tftpd64

- Tera Term: the two serial terminals (all steps).
- tftpd64: a TFTP server for the Linux disk image (Step 5A or 5B).

WSL2 with Debian 13 (only to build images yourself)

1. Open PowerShell and check what is installed: `wsl -l -v`.
2. If Debian is not listed, install it: `wsl --install -d Debian`.
3. Open Debian from the Start menu and create a username and password.
4. Check the version: `cat /etc/os-release` should report Debian GNU/Linux 13 (trixie).

Debian 13 is what this BSP is tested on. Ubuntu 22.04 or 24.04 should also work, but was not tested for this release.

SoftConsole (only for bare-metal programs, Appendix E)

Download it from Microchip's SoftConsole page; keep the default settings and pick any install directory.

Links

- Libero SoC download (FPGA Software Downloads)
  https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/fpga-software-downloads
- Tera Term: https://teratermproject.github.io/index-en.html
- tftpd64: https://pjo2.github.io/tftpd64/

Step 2: Download the release files

Everything comes from the `nmc-bsp` repository on GitHub. Open https://github.com/nextage-llc/nmc-bsp/releases, expand **Assets** under the latest release, and download the files you need into one folder on your PC, for example `C:\nmc-images`.

Which files you need

- Everyone: `nmc-fpga.job` (Step 4)
- Linux with Buildroot (5A): `nmc-buildroot-payload.bin` and `nmc-buildroot.img`
- Linux with Yocto (5B): `nmc-yocto-payload.bin` and `nmc-yocto.wic.gz`
- RTEMS (5C): `nmc-rtems-payload.bin`

`SHA256SUMS` lists the checksum of every file. To check a download in PowerShell, run `Get-FileHash nmc-fpga.job` and compare the hash with its line in `SHA256SUMS`.

Step 3: Cable the board and open the serial terminals

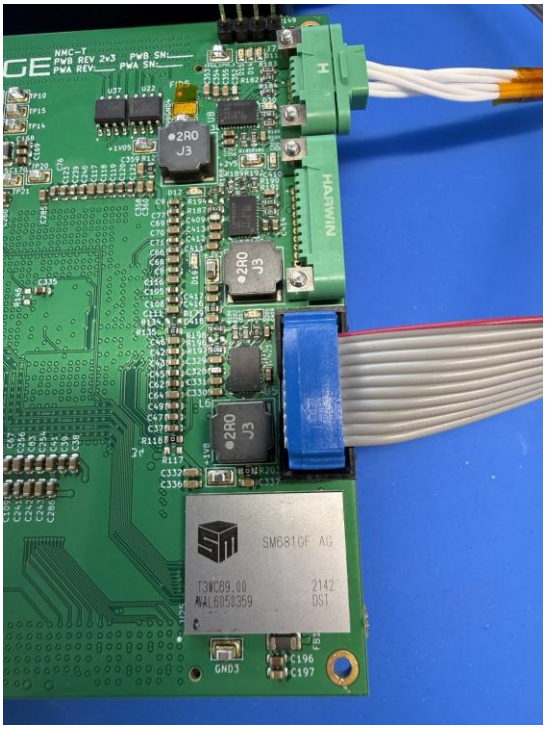
Connect the board to the host PC with USB-to-serial cabling into the UART0 and UART1 headers for the consoles and one Ethernet cable for TFTP, plus a Microchip FlashPro programmer and a 5 V power feed (photos below).

Connection	From	To	Used for	Source detail
Power	5 V supply	Board (J7)	Powering and power-cycling the board	Photo below
Programming link	Host PC (FlashPro Express), then USB to FlashPro5 or FlashPro6	Board (J2)	Programming the FPGA design and firmware (Step 4)	Photos below
USB	Host PC (USB)	Board (UART0 and UART1 headers)	Two serial ports: UART0 (HSS bootloader) and UART1 (U-Boot, Linux and RTEMS console)	Photo below
Ethernet	Host PC	Board (J19)	TFTP transfer of `Linux disk image` (Step 5A or 5B)	Photo below. Direct to the host.

Connect the programmer. Plug the FlashPro5 or FlashPro6 into a USB port on the host PC, then plug its ribbon cable into the J2 connector on the NMC's edge.

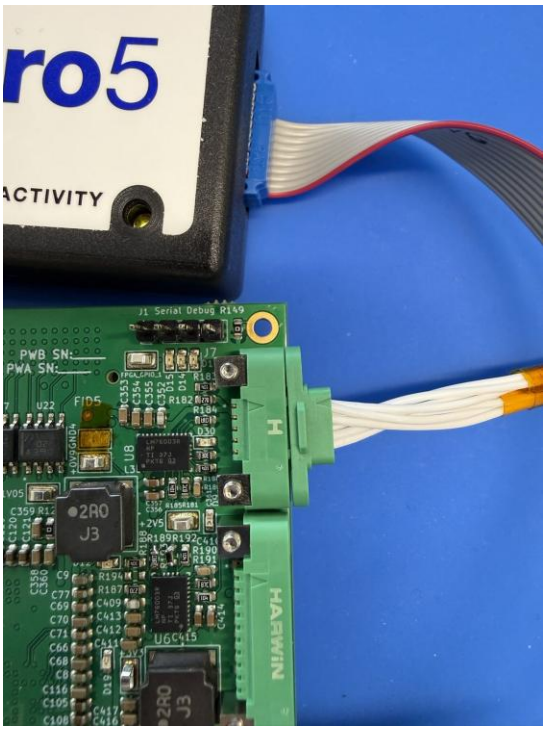


The FlashPro5 (a FlashPro6 works the same way). The USB cable to the host PC leaves on the left, and the ribbon cable to the NMC leaves on the right.



The ribbon cable seats in the J2 connector on the board edge. Match the orientation in the photo, with the cable's red-stripe edge on top.

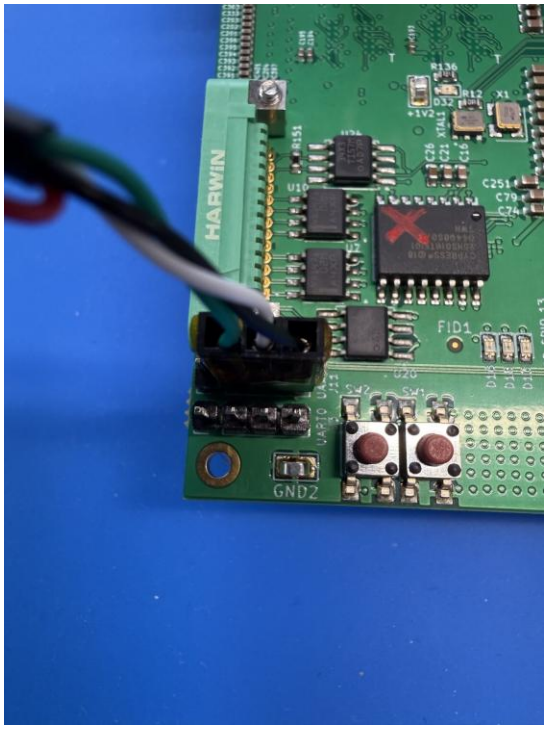
Connect power. Feed 5V into connector J7 on the board edge. Set the supply's current limit to 2 A. There are no jumpers or boot-mode settings to set and no power-up order: turn on the supply and the board sequences its own internal rails.



Power goes into J7, shown here with the white wire bundle plugged in.

Connect the serial consoles. Connect the USB-to-serial cable to the board's UART headers, UART0 for the HSS bootloader and UART1 for U-Boot, Linux and RTEMS, then plug the USB end into the host PC.

Connect Ethernet. Plug the Ethernet cable into the board's J19 connector, and run the other end to the host PC.



Serial consoles. The UART headers sit together near the board's lower-left corner: UART0 (J3) and UART1 (J11). The photo shows a USB-to-serial connector seated on UART1.

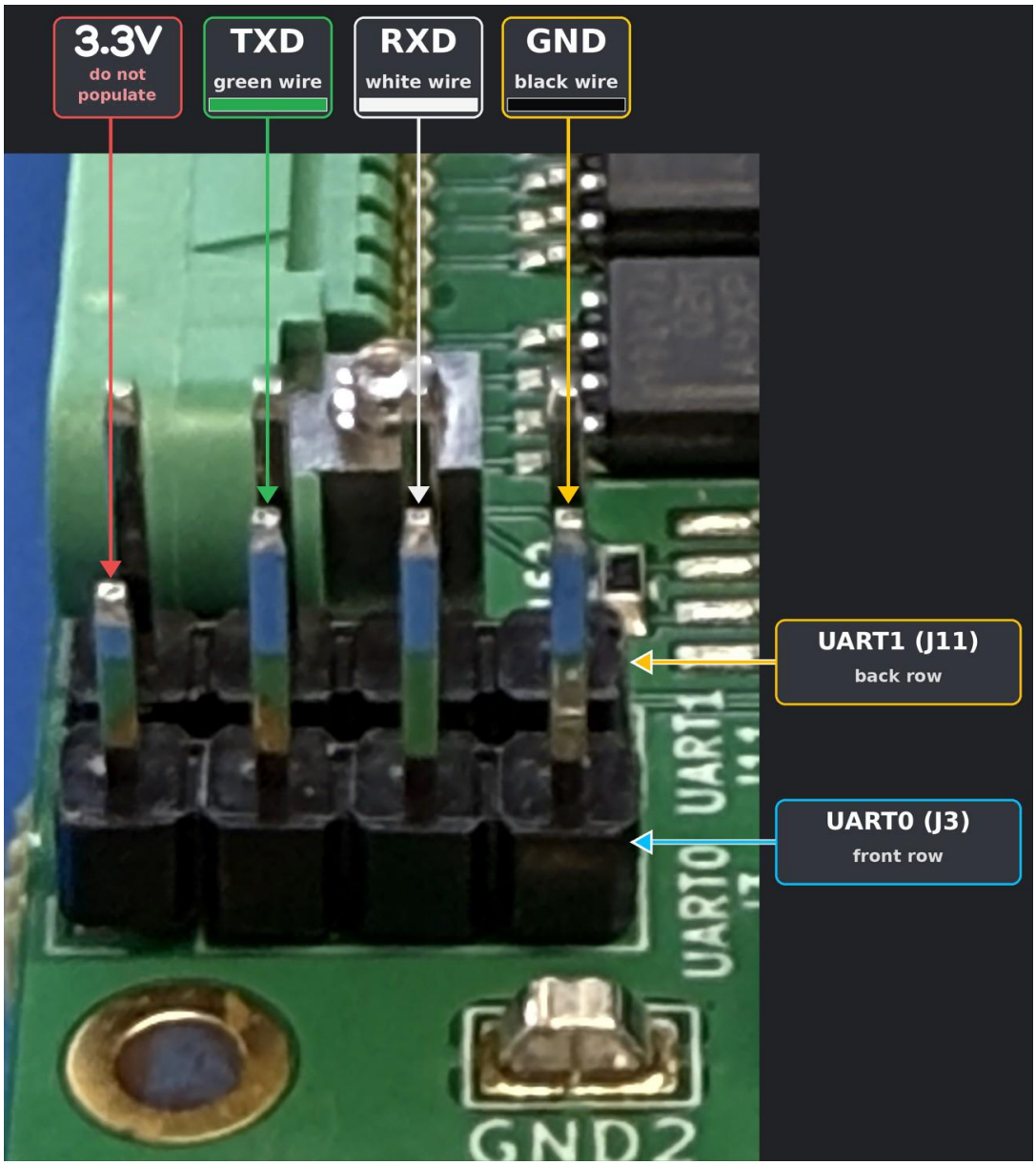


Ethernet. The cable goes into connector J19 at the board edge, next to the two HALO magnetics modules.

Serial cable wiring. The cable is a 3.3 V TTL USB-to-UART cable (PL2303TA, USB A male, 1x4 female 2.54 mm socket). It carries one UART, so use one cable per UART header. The board header takes only three of the four wires, so leave the red 5 V wire out of the socket.

Header position (left to right, as photographed)	Board signal	Cable wire	Note
1	3.3V	Red (VCC)	Do not populate
2	TXD	Green	Cable RXD
3	RXD	White	Cable TXD
4	GND	Black	Cable GND

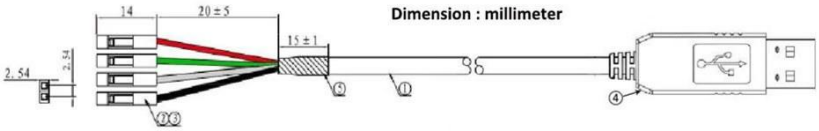
Signals are named from the board's side, so the board's TXD meets the cable's RXD.



The two UART headers, labeled. Both use the same pin order, left to right as photographed with the board edge at the bottom: 3.3V (leave unpopulated), TXD, RXD, GND. The back row is UART1 (J11) and the front row is UART0 (J3).

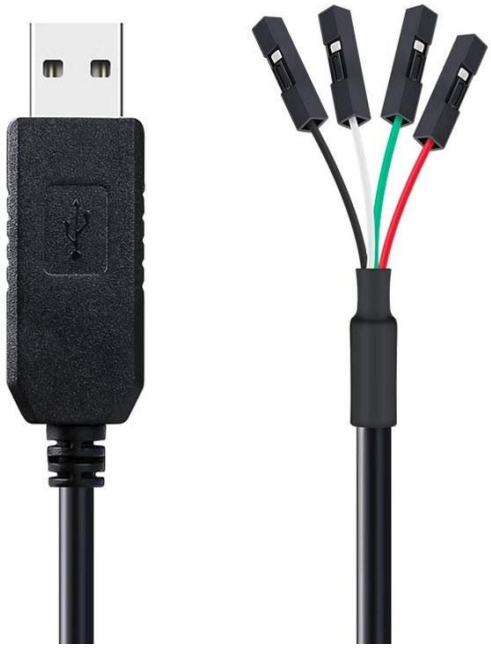


The cable's socket seated on a UART header. The green, white, and black wires sit on pins 2 to 4, and the leftmost pin (3.3V) is left open.



① PVC Jacket
② 2.54mm pitch socket
③ 2.54mm 1*4p terminal ,female
④ USB A male, PL2303TA TTL 3.3V PCBA inside
⑤ Heat shrink tube, black, OD=5.0

1*4P Female Socket	Name	Colour	Description
Pin 1	TXD	White	Transmit Asynchronous Data
Pin 2	RXD	Green	Receive Asynchronous Data
Pin 3	GND	Black	Device ground supply
Pin 4	VCC	Red	+5V



The USB-to-UART cable and its own pin table. Its socket pins are numbered differently from the board header, so use the table above for placement.

Open two Tera Term windows, one per COM port that the USB connection creates. It is also acceptable to use one Tera Term window and just swap between the 2 UARTs. Tera Term will automatically pick up the 2nd UART comms without any intervention.

Use these settings on both:

- Baud: 115200
- Data: 8 bit
- Parity: none
- Stop: 1 bit
- Flow control: none

Window	COM port	Shows
Tera Term Window 1 (UART0)	Port showing the HSS boot log	UART0: HSS bootloader, hardware initialization and boot logs
Tera Term Window 2 (UART1)	The other port	UART1: OpenSBI banner, U-Boot prompt, then the Linux or RTEMS console

Which port is which. Windows assigns the COM numbers itself, and they differ between PCs and USB hubs, so identify the ports by what they print once the board is programmed (Step 4). Open both, then reset the board. The window that prints the HSS boot messages is UART0 (Window 1). The other is UART1 (Window 2), which prints DDR training and the OpenSBI banner, then the U-Boot prompt.

Step 4: Program the FPGA design and HSS bootloader

`nmc-fpga.job` holds the FPGA design and the HSS bootloader (stored in the eNVM). Program it once on a new board, and again whenever the FPGA design or the HSS changes.

1. Connect the programmer to J2 and to the PC by USB, then power the board (Step 3).
2. In FlashPro Express choose **Project → New Job Project from FlashPro Express Job**, select `nmc-fpga.job`, pick a folder for the project, and click **OK**.
3. Check that the programmer is listed, leave the action on **PROGRAM**, and click **RUN**. Programming takes a few minutes and ends with **RUN PASSED**.
4. Disconnect the programmer and power cycle the board.

Checkpoint: Tera Term Window 1 (UART0) prints the HSS boot banner. Check for `BOARD=NMC2v3` at the end of the version line, `DESIGNID: NMC` and `DESIGNVER: 2c01`. The Build ID matches the one in the CHANGELOG. Until Step 5 puts a payload on the eMMC, the HSS stops after "Trying to get boot image via MMC"; that is expected.

To change the FPGA design, see Appendix D.

Step 5: Load an operating system

The HSS boots whatever payload is on the eMMC. Every OS starts the same way: flash its payload over YMODEM (below). Linux then also needs its disk image, flashed over TFTP. Then continue with 5A, 5B or 5C.

Flash the payload and the disk image from the same build: the Buildroot and Yocto U-Boots look for the kernel in different places.

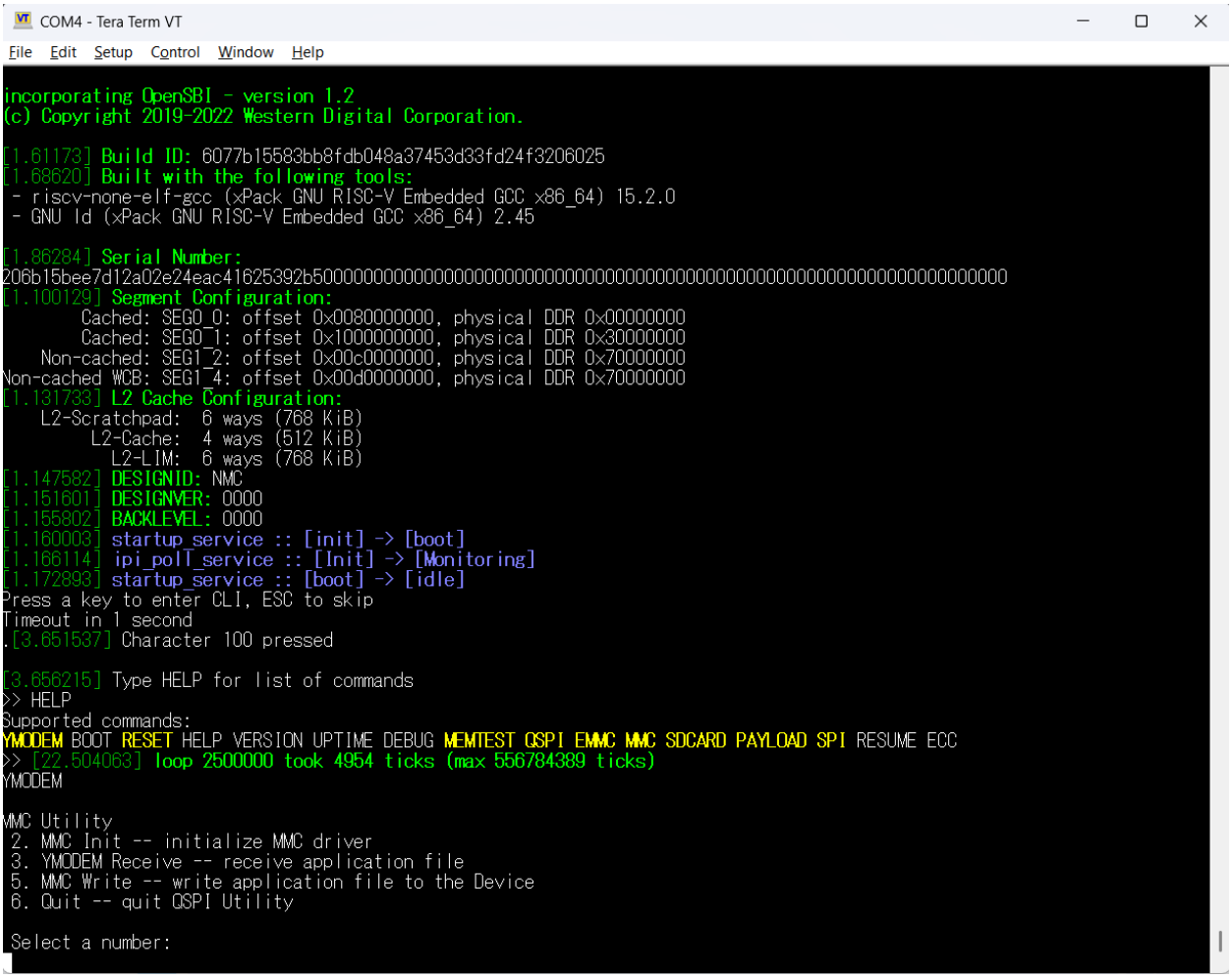
Flash a payload to eMMC (YMODEM, UART0)

Get to the HSS prompt

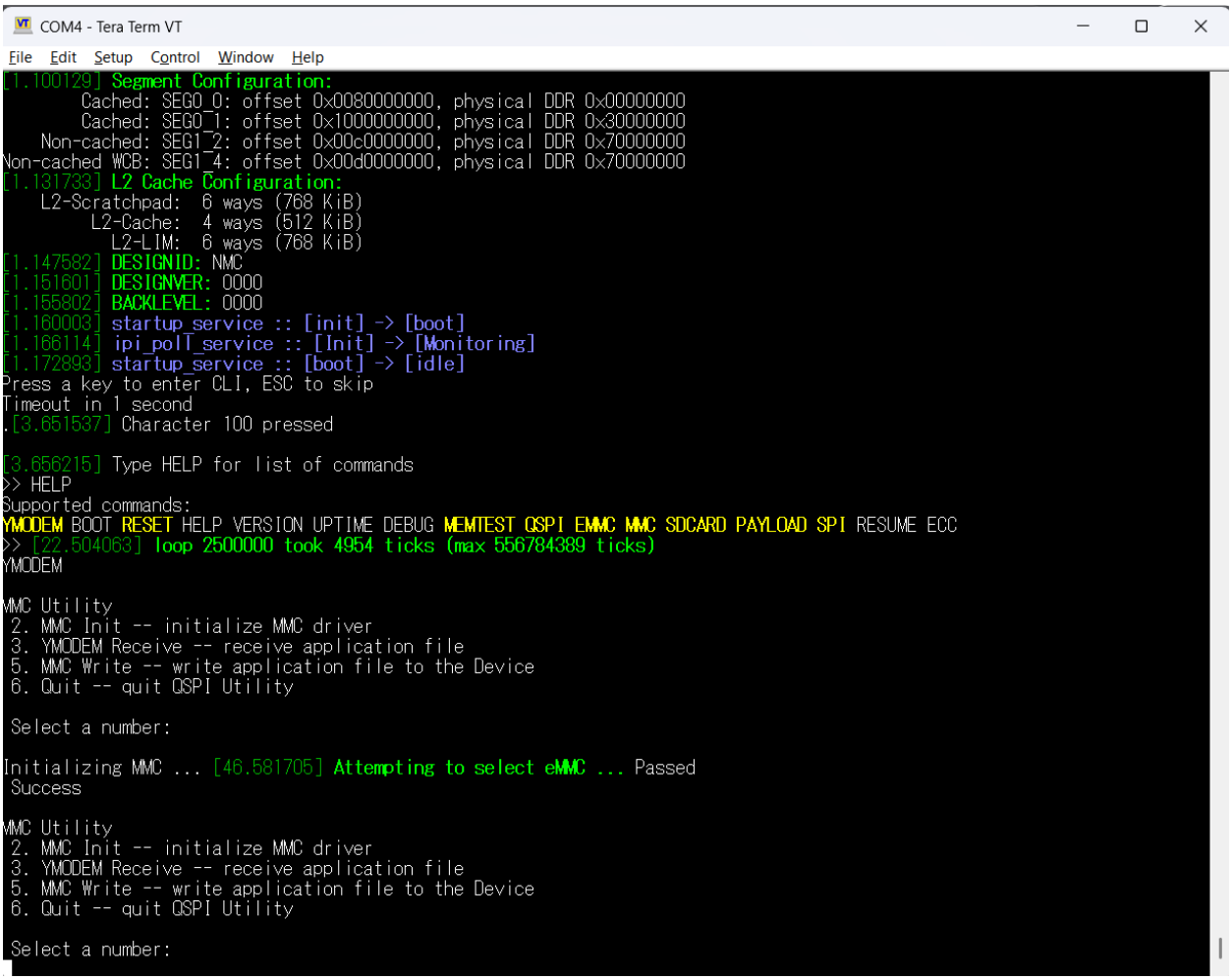
1. Power cycle the board or press its Reset button.
2. Immediately press any key in Tera Term Window 1 (UART0) to stop auto-boot and reach the >> prompt.

Receive the image

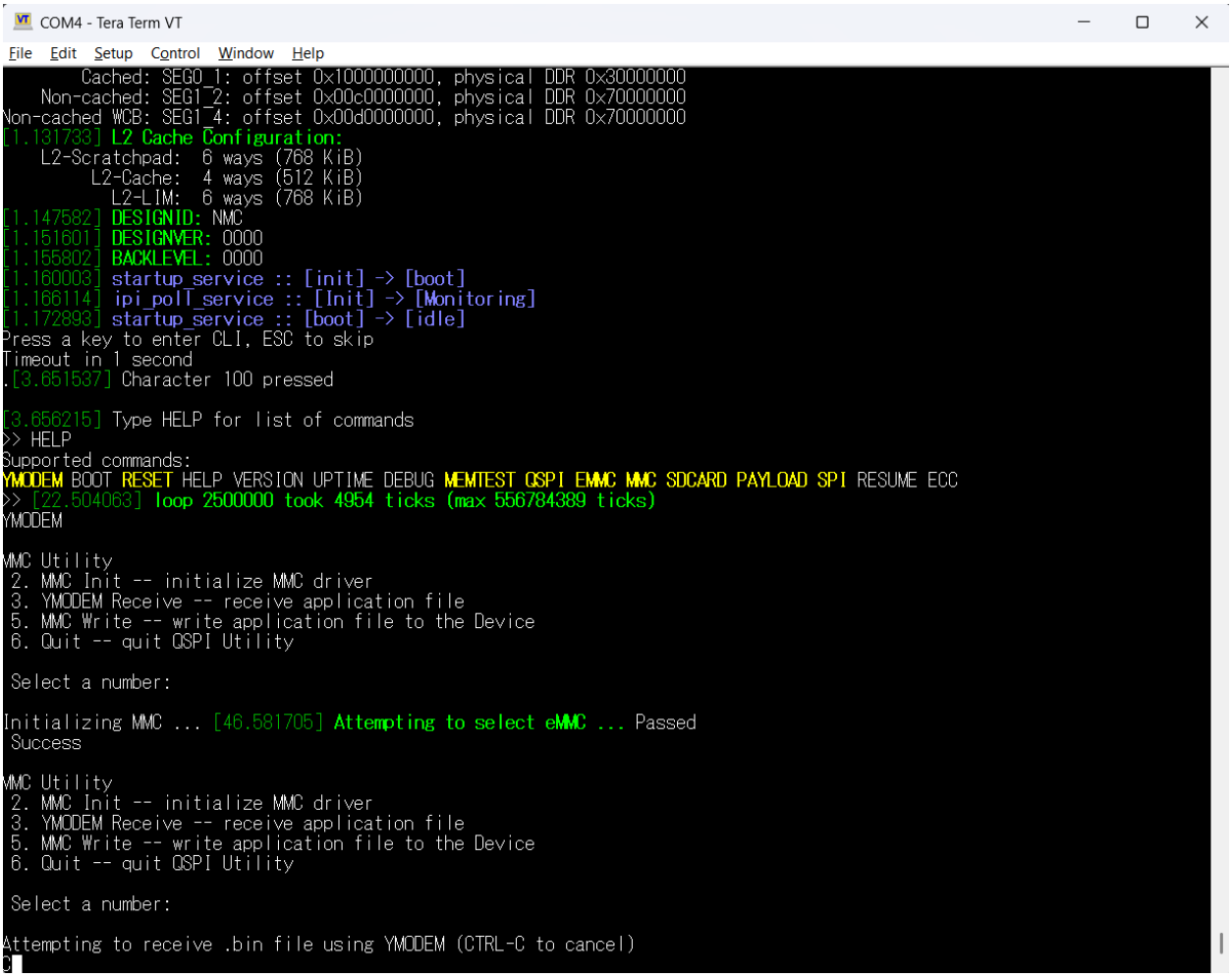
1. Type `YMODEM` and press Enter.



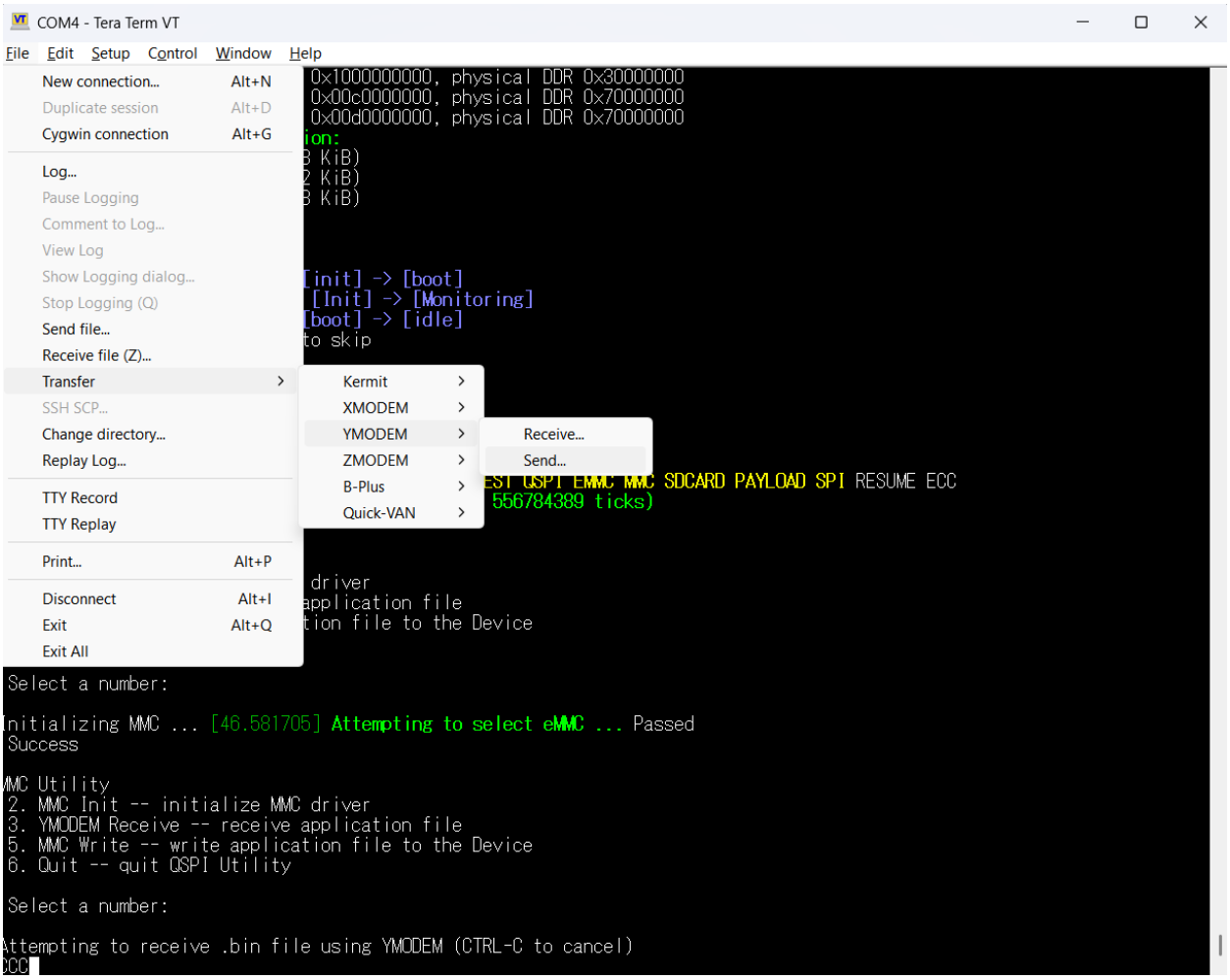
2. Type 2 and press Enter to run **MMC Init**.



3. Type 3 and press Enter to select **YMODEM Receive**.



4. In Window 1 (UART0), choose **File → Transfer → YMODEM → Send...**, select the payload for your OS (for example `nmc-buildroot-payload.bin`), and click **Open**.



5. Wait for the transfer to finish.

Write it permanently

1. Type 5 and press Enter to run **MMC Write**.
2. Type 6 and press Enter to quit the utility.

5A: Linux with Buildroot

Payload `nmc-buildroot-payload.bin`, disk image `nmc-buildroot.img`. To build them yourself, see Appendix A.

1. Flash the payload

Flash `nmc-buildroot-payload.bin` as above.

Checkpoint: after a reset, you should land at a U-Boot prompt in Tera Term Window 2 (UART1). If not, resolve that before flashing the disk image.

Both windows should look like this after the payload is written and the board resets, before you load the Linux image next.

UART0 (Window 1): the HSS boot log. It starts with HSS decompressing from eNVM, then shows the Microchip banner and the HSS version and build details.

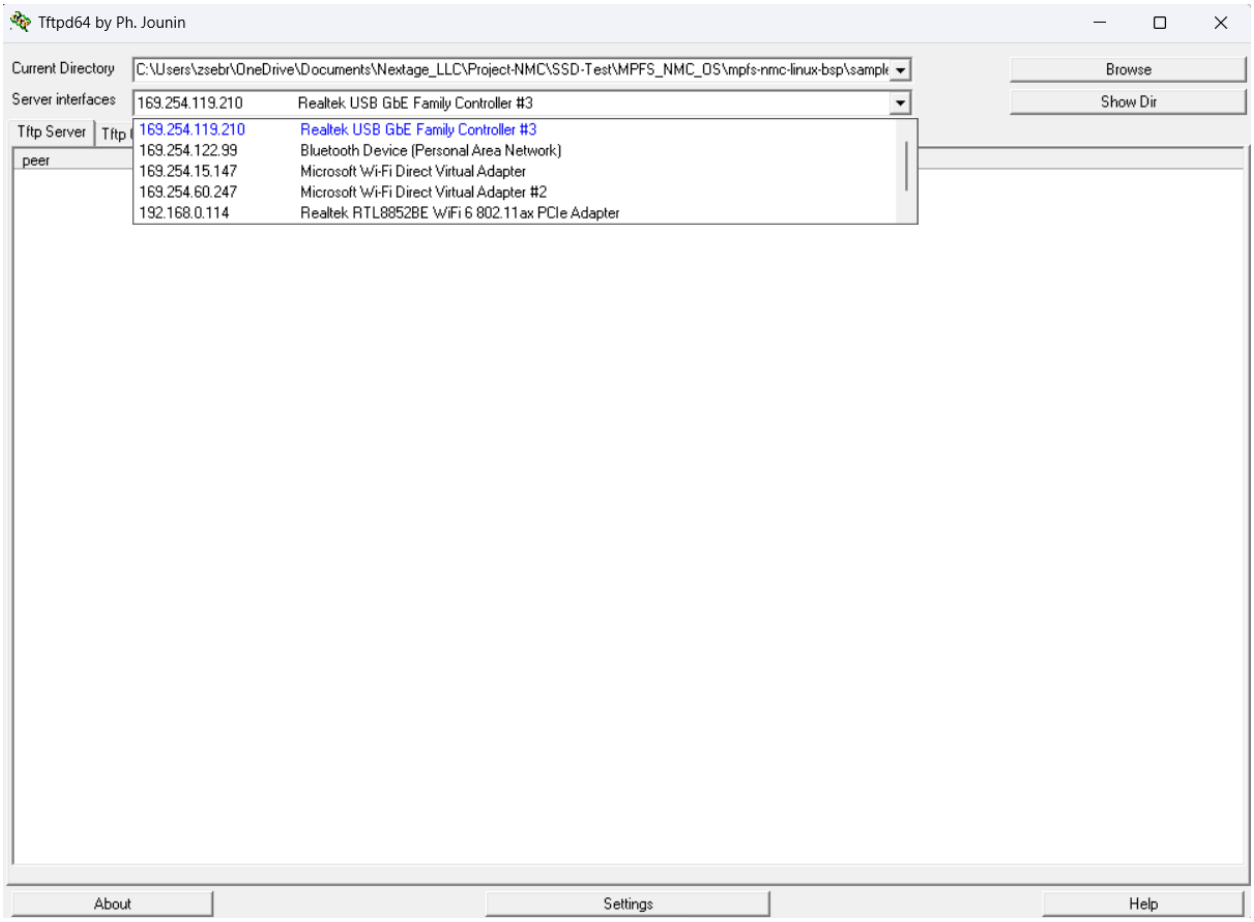
UART1 (Window 2): DDR training, then the OpenSBI banner with platform and domain details, as U-Boot starts.

2. Flash the disk image (TFTP, UART1)

Serve the disk image from the PC over the Ethernet cable, load it into RAM from U-Boot, and write it to eMMC.

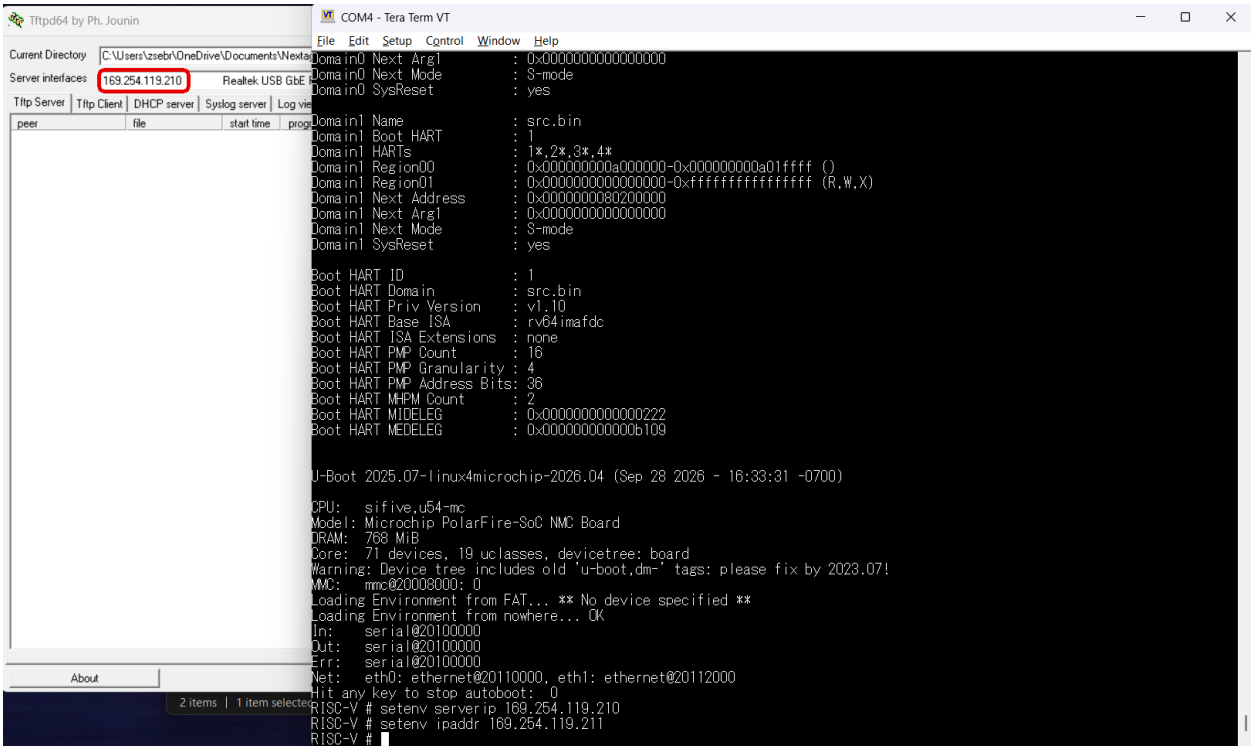
Set up the TFTP server on the host

1. Confirm the Ethernet cable runs from the board's J19 connector to the host PC. Use the eth0 end of the cable.
2. Start a TFTP server such as tftpd64. Point it at the folder containing `the disk image (C:\nmc-images)` and select the host's Ethernet interface.
3. Use tftp64 to find your Ethernet controller on your machine and write down the IP address. You'll need it for the next steps. In this picture, the IP address of the ethernet port is 169.254.119.210. Yours will be different.



Set the network in Tera Term UART 1 (U-Boot)

```
setenv serverip 169.254.69.98
setenv ipaddr 169.254.69.99
```



Set `serverip` to your host's Ethernet interface address, and `ipaddr` to an unused address on the same subnet. Set the host's Ethernet adapter to the `serverip` address manually.

Load and write the image

```
tftpboot 0x84000000 nmc-buildroot.img
setexpr blkcnt ${filesize} + 0x1ff
setexpr blkcnt ${blkcnt} / 0x200
mmc dev 0
mmc write 0x84000000 0 ${blkcnt}
```

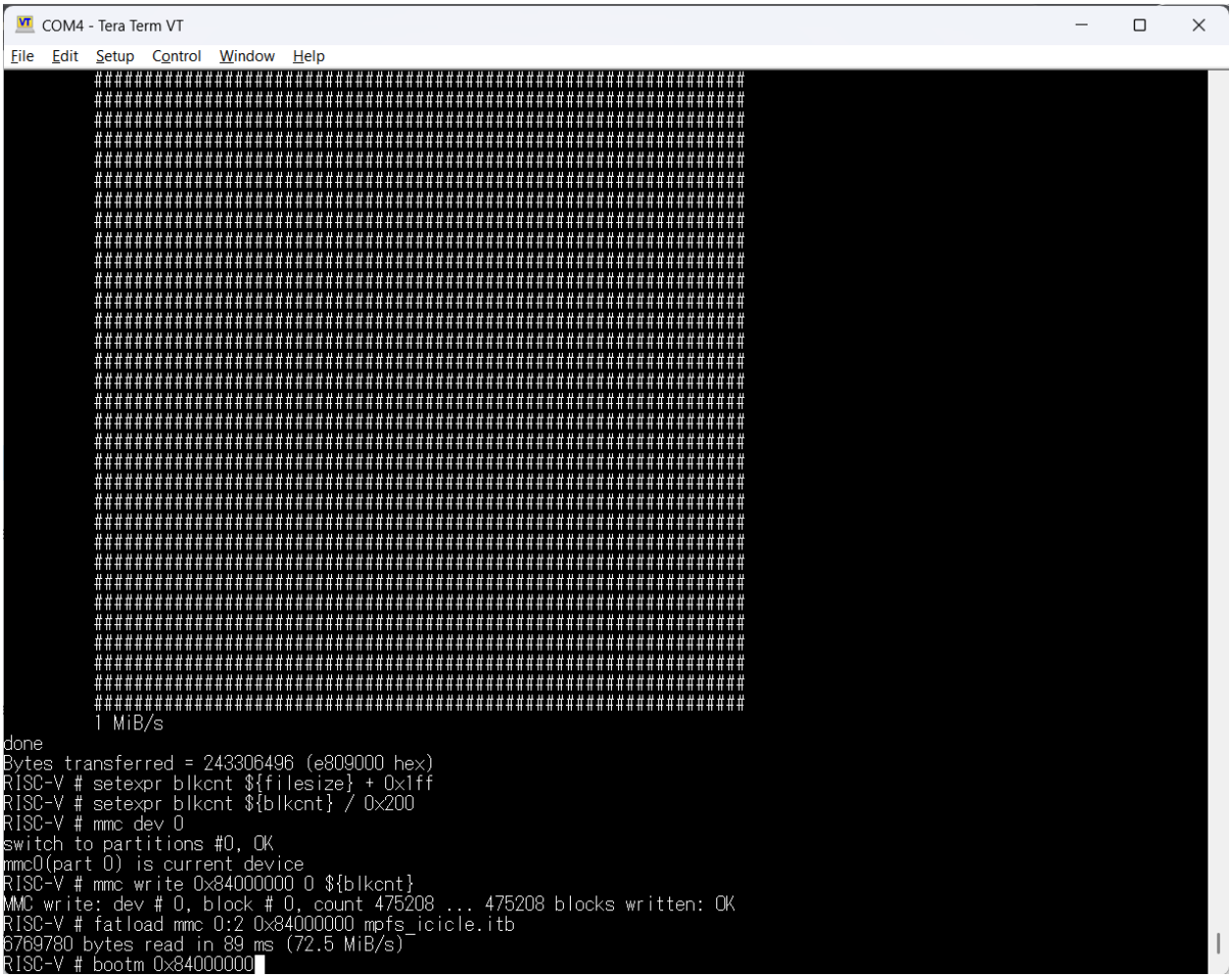
(0x84000000 has 6 0s after the 84)

The two `setexpr` lines convert the downloaded file size into a count of 512-byte blocks, rounded up, for `mmc write`.

3. Boot Linux

Boot manually

```
fatload mmc 0:2 0x84000000 mpfs_icicle.itb
bootm 0x84000000
```



Once U-Boot is set to load from eMMC by default, typing `boot` or power cycling also works.

Log in. Username `root`, password `root`.

Persist auto-boot. If a reset drops you back at the U-Boot prompt, `bootcmd` has not been saved. Check it:

```
printenv bootcmd
# bootcmd=fatload mmc 0:2 0x84000000 mpfs_icicle.itb; bootm 0x84000000
```

If it matches the line above, run:

```
saveenv
```

Resets will then boot directly into Linux.

5B: Linux with Yocto

Payload `nmc-yocto-payload.bin`, disk image `nmc-yocto.wic.gz`. To build them yourself, see Appendix B.

1. Flash the payload

Flash `nmc-yocto-payload.bin` as above. After a reset, Window 2 (UART1) should stop at a U-Boot prompt.

2. Flash the disk image (TFTP, UART1)

Set up tftpd64 and `serverip`/`ipaddr` as in 5A. Then load the compressed image and write it with `gzwrite`, which decompresses it while writing, so only the compressed file has to fit in RAM:

```
tftpboot 0x84000000 nmc-yocto.wic.gz
mmc dev 0
gzwrite mmc 0 0x84000000 ${filesize}
```

`gzwrite` prints the bytes written (721,726,464 for the 2026.10 image). Always send the `.wic.gz`: the uncompressed `.wic` does not fit in U-Boot's RAM. Then re-read the eMMC and check the result:

```
mmc rescan
part list mmc 0
ls mmc 0:1
```

`part list` should show three partitions named `boot`, `primary` and `root`, and `ls` should list `boot.scr`, `fitImage` and `uboot.env`.

3. Boot Linux

Reset the board. It boots Linux by itself: U-Boot runs `boot.scr` from the boot partition, so there is no `bootcmd` to save. If it stops at the U-Boot prompt, boot manually:

```
fatload mmc 0:1 0x84000000 fitImage
bootm 0x84000000
```

Log in. Username `root`, no password; you are asked to set a new password at the first login.

5C: RTEMS

Payload `nmc-rtems-payload.bin`: the RTEMS sample application `ticker`, built with the NMC device tree. RTEMS needs no disk image or Ethernet. To build RTEMS or your own application, see Appendix C.

1. Flash `nmc-rtems-payload.bin` as above.
2. Type `boot` in Window 1 (UART0), or power cycle the board.

Window 1 shows the HSS loading the payload. "GPT signature not as expected … using offset 0" is normal: the payload is written to the start of the eMMC, without a partition table.

Window 2 (UART1) shows `*** BEGIN OF TEST CLOCK TICK ***`, then lines from tasks TA1, TA2 and TA3 every 5, 10 and 15 seconds with the advancing time, and after about 35 seconds `*** END OF TEST CLOCK TICK ***`. The `[ RTEMS shutdown ]` block after it is how an RTEMS test exits.

Appendix A: Build the Linux images with Buildroot

Builds `nmc-buildroot-payload.bin` and `nmc-buildroot.img` from source, in WSL2 (Step 1). The first build takes about an hour. Work inside the Linux filesystem (e.g. `~/nmc-buildroot`), not under `/mnt/c/...`, which is much slower and causes path problems. Same steps as `nmc-linux-bsp/buildroot/README.md`.

1. Install the dependencies

```
sudo apt-get install subversion build-essential bison flex gettext \
  libncurses5-dev texinfo autoconf automake libtool mercurial git-core \
  gperf gawk expat curl cvs libexpat-dev bzr unzip bc python3-dev \
  wget cpio rsync xxd bmap-tools libgnutls28-dev libgl1
```

2. Get Buildroot, Microchip's external tree and this BSP

```
mkdir -p ~/nmc-buildroot && cd ~/nmc-buildroot
git clone https://git.busybox.net/buildroot -b 2025.02.11
git clone https://github.com/linux4microchip/buildroot-external-microchip.git
git -C buildroot-external-microchip checkout ebdcfd7
git clone https://github.com/nextage-llc/nmc-bsp.git
```

Buildroot rejects environment variables that contain spaces, and WSL2 adds the Windows `PATH`. If `make` complains, run `export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin` first.

3. Build and copy the images to Windows

```
cd ~/nmc-buildroot/buildroot
make BR2_EXTERNAL=../buildroot-external-microchip:../nmc-bsp/nmc-linux-bsp/buildroot
nmc_defconfig
make
mkdir -p /mnt/c/nmc-images
cp output/images/payload.bin /mnt/c/nmc-images/nmc-buildroot-payload.bin
cp output/images/sdcard.img /mnt/c/nmc-images/nmc-buildroot.img
```

The images are in `output/images/`; flash them as in Step 5A. Later builds are just `make`.

4. Rebuild after a change

Run from `~/nmc-buildroot/buildroot`, then copy the images again:

- Device tree (`nmc-linux-bsp/linux/*.dts`, `nmc-linux-bsp/uboot/*.dts`): `make linux-rebuild uboot-rebuild host-hss-payload-generator-rebuild all`
- Kernel configuration (`nmc-linux-bsp/linux/nmc_defconfig`): `make linux-reconfigure udmabuf-rebuild all`
- U-Boot configuration (`nmc-linux-bsp/uboot/nmc_defconfig`): `make uboot-reconfigure host-hss-payload-generator-rebuild all`
- Packages or root filesystem (`nmc-linux-bsp/buildroot/configs/nmc_defconfig`): `make nmc_defconfig`, then `make`

If `make` reports `Clock skew detected`, the WSL2 clock has drifted: run `wsl --shutdown` in PowerShell, reopen WSL and run `make` again.

Appendix B: Build the Linux images with Yocto

Builds `nmc-yocto-payload.bin` and `nmc-yocto.wic.gz` from source, in WSL2 (Step 1). Needs about 100 GB of disk and 16 GB RAM; the first build takes one to two hours. Same steps as `nmc-linux-bsp/yocto/README.md`.

1. Install the dependencies and Google's repo tool

```
sudo apt update
sudo apt-get install gawk wget git git-lfs diffstat unzip texinfo gcc \
  build-essential chrpath socat cpio python3 python3-pip \
  python3-pexpect xz-utils debianutils iputils-ping python3-git \
  python3-jinja2 python3-subunit zstd lz4 file locales \
  libacl1 libgl1 bmap-tools
sudo locale-gen en_US.UTF-8
mkdir -p ~/.local/bin
curl -fsSL https://storage.googleapis.com/git-repo-downloads/repo -o ~/.local/bin/repo
chmod a+rx ~/.local/bin/repo
```

Open a new terminal (so `~/.local/bin` is on your PATH), then check that `repo version` works.

2. Get Microchip's Yocto layers and this BSP

Set your git identity (`repo init` needs it; skip if already set):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git lfs install
mkdir -p ~/nmc-yocto && cd ~/nmc-yocto
repo init -u https://github.com/linux4microchip/meta-mchp-manifest.git \
  -b refs/tags/linux4microchip-2026.04 -m polarfire-soc/default.xml
repo sync
git clone https://github.com/nextage-llc/nmc-bsp.git
```

3. Set up the build and build

```
cd ~/nmc-yocto
TEMPLATECONF=$PWD/nmc-bsp/nmc-linux-bsp/yocto/meta-nmc/conf/templates/nmc \
  source openembedded-core/oe-init-build-env
bitbake-layers show-layers | grep meta-nmc
```

The last command should list `meta-nmc`. Then build:

```
bitbake mchp-base-image
```

Look for `all succeeded` in the `Tasks Summary` line near the end; warnings like `Failed to fetch URL …, attempting MIRRORS` are normal (BitBake then downloads from a mirror). In every new shell, run `cd ~/nmc-yocto && source openembedded-core/oe-init-build-env` before `bitbake`.

4. Copy the images to Windows

```
mkdir -p /mnt/c/nmc-images
cd ~/nmc-yocto/build/tmp-glibc/deploy/images/mpfs-icicle-kit
cp payload.bin /mnt/c/nmc-images/nmc-yocto-payload.bin
cp mchp-base-image-mpfs-icicle-kit.rootfs.wic.gz /mnt/c/nmc-images/nmc-yocto.wic.gz
```

Then flash them as in Step 5B.

If the build stops with do_package errors mentioning Bad address

The host `tar` is failing under Yocto's fake-root tool (seen after an Ubuntu 22.04 `tar` update). Use the `tar` from Yocto's prebuilt host tools, then run `bitbake mchp-base-image` again:

```
cd ~/nmc-yocto
./openembedded-core/scripts/install-buildtools
source ~/nmc-yocto/openembedded-core/buildtools/environment-setup-x86_64-pokysdk-linux
source openembedded-core/oe-init-build-env
rm -rf ~/nmc-yocto/build/tmp-glibc/hosttools
tar --version | head -1
```

The last command should report tar 1.35. In every new shell, run both `source` lines (buildtools first) before `bitbake`.

Appendix C: Build RTEMS and make your own payload

In WSL2 (Step 1). Same steps as `nmc-rtems-bsp/README.md`. To make a payload from the sample `ticker` (already in `nmc-rtems-bsp/sample/`) or from your own RTEMS application, steps 1 and 4 are enough; steps 2 and 3 build the RTEMS toolchain (about 30 minutes) and the RTEMS BSP with the NMC device tree.

1. Get this BSP and build the HSS payload generator

```
sudo apt-get install build-essential device-tree-compiler libyaml-dev \
  libelf-dev libssl-dev git python3 python-is-python3 bison flex \
  texinfo unzip bzip2 xz-utils
mkdir -p ~/nmc-rtems && cd ~/nmc-rtems
git clone https://github.com/nextage-llc/nmc-bsp.git
make -C nmc-bsp/hart-software-services/tools/hss-payload-generator
```

2. Build the RTEMS 6.1 toolchain

```
cd ~/nmc-rtems
git clone --branch 6.1 https://gitlab.rtems.org/rtems/tools/rtems-source-builder.git rsb
cd rsb/rtems
../source-builder/sb-check
../source-builder/sb-set-builder --prefix=$HOME/nmc-rtems/6 6/rtems-riscv
export PATH=$HOME/nmc-rtems/6/bin:$PATH
```

In every new shell, run the `export PATH` line again.

3. Build the RTEMS BSP with the NMC device tree

```
cd ~/nmc-rtems
git clone --branch 6.1 https://gitlab.rtems.org/rtems/rtos/rtems.git
cd rtems
cat > config.ini << EOF
[riscv/mpfs64imafdc]
RTEMS_POSIX_API = True
RTEMS_SMP = True
BUILD_SAMPLES = True
BSP_DTB_IS_SUPPORTED = True
BSP_DTB_HEADER_PATH = $HOME/nmc-rtems/nmc-bsp/nmc-rtems-bsp/dts/mpfs-nmc-rtems-dtb.c
BSP_START_COPY_FDT_FROM_U_BOOT = False
EOF
./waf configure --prefix=$HOME/nmc-rtems/6
./waf
./waf install
```

The sample is built at `build/riscv/mpfs64imafdc/testsuites/samples/ticker.exe`; your own applications link against the BSP installed in `~/nmc-rtems/6`.

4. Make the payload

`sample/` already holds `ticker`. For your own application, copy its ELF next to `rtems-payload.yaml` and change `ticker` under `payloads:` in `rtems-payload.yaml` to its name. Then:

```
cd ~/nmc-rtems/nmc-bsp/nmc-rtems-bsp/sample
../../hart-software-services/tools/hss-payload-generator/hss-payload-generator -c rtems-
payload.yaml nmc-rtems-payload.bin
```

Copy `nmc-rtems-payload.bin` to `C:\nmc-images` (`cp nmc-rtems-payload.bin /mnt/c/nmc-images/`) and flash it as in Step 5C.

Appendix D: Change the FPGA design

Optional: for adding your own logic to the FPGA, removing blocks, or changing the MSS (the processor subsystem's peripherals and pins). The Libero project is generated by a Tcl script in the repository, the same way as Microchip's PolarFire SoC reference designs. Same steps as `fpga/README.md`.

Libero license

1. Libero needs a license tied to your PC to build designs (programming with FlashPro Express does not). Generate it from Microchip's FPGA Software Licenses page and follow the Libero SoC License Installation Guide for the 1-year node-locked Silver license, which is free.
2. Node-locked means the license is bound to this one PC. Floating licenses are for servers.
3. Once the license is installed and linked, Libero SoC launches and runs.

Generate the Libero project

1. Clone the repository to a short path, because Libero needs the `fpga` folder path under 90 characters: `git clone https://github.com/nextage-llc/nmc-bsp.git C:\nmc-bsp`.
2. In Libero SoC 2025.2 choose **Project → Execute Script**, select `C:\nmc-bsp\fpga\nmc-fpga.tcl`, and click **Run**. It creates the project in `fpga\NMC2v3`, runs the design flow and adds the HSS to the eNVM (about 15 minutes; the first run also downloads the IP cores). The project then matches `nmc-fpga.job`.
3. Make your changes in the `NMC` SmartDesign, run the design flow again, use **Export FlashPro Express Job**, and program the board as in Step 4 (or directly from Libero with **Run PROGRAM Action**).

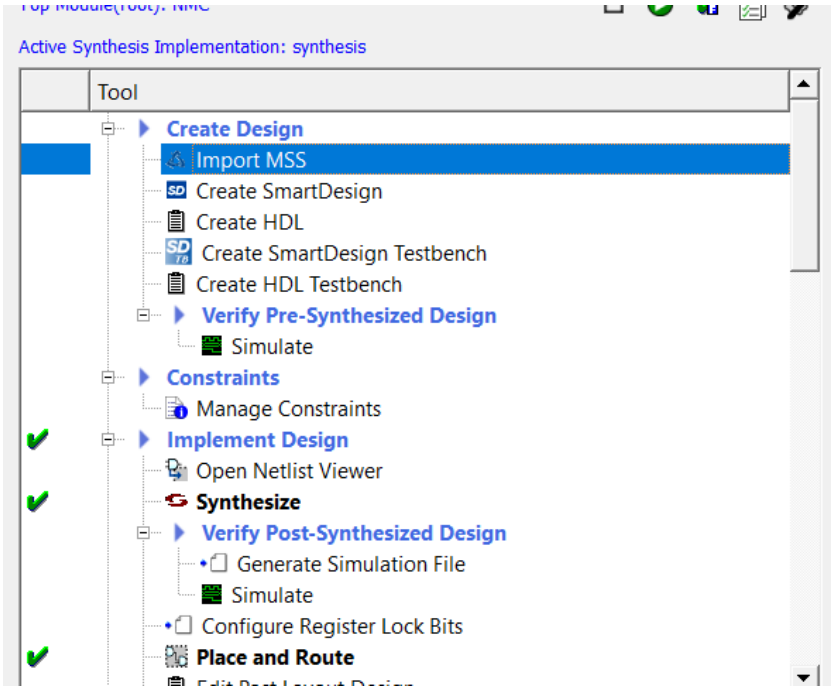
Free for your own blocks: FIC3 slots 5–15 (`0x5500_0000`–`0x5F00_0000`) and fabric interrupts F2M[6–58] (PLIC 124–176). The full fabric memory map is in `fpga/README.md`. If you add or change peripherals, update the Linux and RTEMS device trees to match.

Known issue: with a design built from the script, the HSS prints `HSS_IHCInit() returned 0` at power-up. Nothing in this release uses the IHC; see the CHANGELOG.

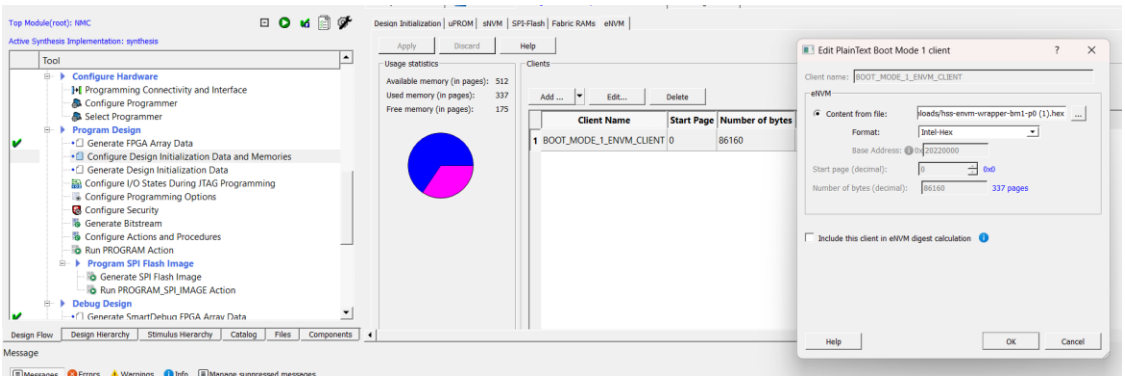
Which file goes where

The MSS Configurator produces files that both Libero and the HSS use; the `.hex` is a program built for the eNVM.

- `fpga/mss/NMC2v3_MSS.cfg`: the MSS settings. Open it in the PolarFire SoC MSS Configurator (installed with Libero), change it, and click **Generate** to write the `.cxz` and `.xml` next to it.
- `NMC2v3_MSS.cxz`: goes into Libero. The script imports it; in an existing project use **Create Design → Import MSS**.
- `NMC2v3_MSS_mss_cfg.xml`: goes into the HSS (`hart-software-services/boards/NMC2v3/soc_fpga_design/xml/`) and into bare-metal SoftConsole projects (Appendix E). Rebuild the HSS after changing it.
- `.hex` (eNVM image, normally the HSS): goes into Libero under **Configure Design Initialization Data and Memories → eNVM**.



Libero design flow: Create Design → Import MSS is where the `.cxz` goes.



The `.hex` import location: Program Design → Configure Design Initialization Data and Memories → eNVM, then Add a client and select the Intel-Hex file.

Rebuild the HSS (Windows)

1. Install the xPack GNU RISC-V Embedded GCC 15.2.0 (e.g. to `C:\Microchip\xpack-riscv-none-elf-gcc-15.2.0-1`), SoftConsole (for its `build_tools`) and Python 3.
2. Build in PowerShell:

```
$env:PATH = "C:\Microchip\xpack-riscv-none-elf-gcc-15.2.0-1\bin;C:\Microchip\SoftConsole-v2022.2-RISC-V-747\build_tools\bin;" + $env:PATH
cd C:\nmc-bsp\hart-software-services
make BOARD=NMC2v3
```

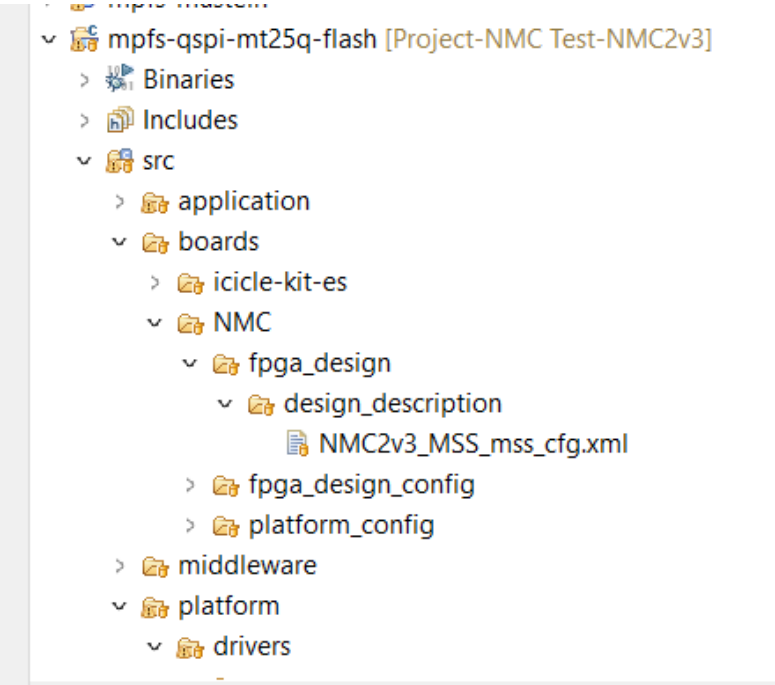
3. Copy `build\bootmode1\hss-envm-wrapper-bm1-p0.hex` over `fpga\script_support\nmc-hss.hex` and generate the project again, or in an existing project point the eNVM client at the new `.hex` (picture above). Export the `.job` and program the board as in Step 4.

Appendix E: Bare-metal programs with SoftConsole

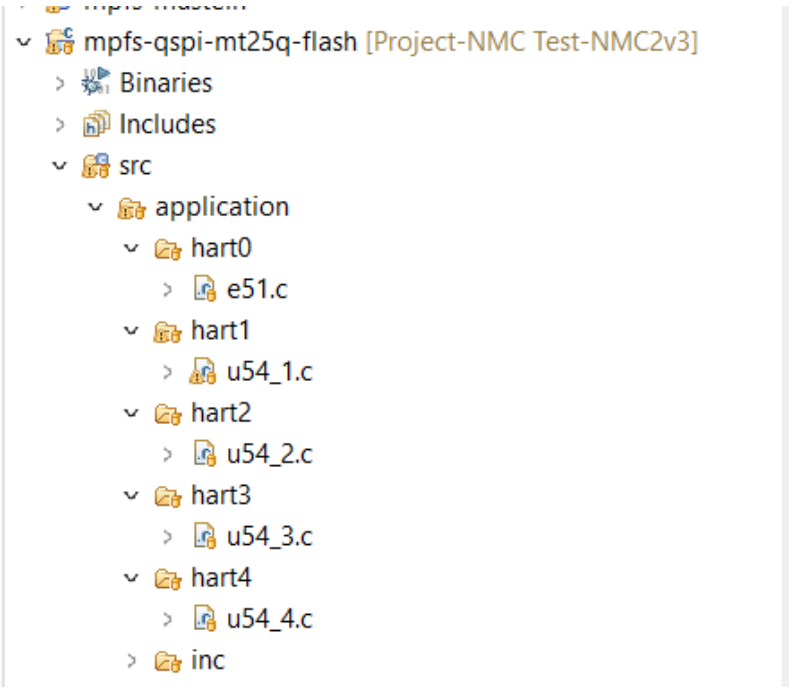
Optional: SoftConsole builds programs that run directly on the processor cores, without Linux or RTEMS. Not tested with this release.

1. Download SoftConsole from Microchip's SoftConsole page; keep the default settings and pick any install directory.
2. Start from one of Microchip's bare-metal example projects (https://github.com/polarfire-soc/polarfire-soc-bare-metal-examples) and import it into SoftConsole.
3. Replace the example's MSS configuration with the NMC2v3 one: drag `fpga/mss/NMC2v3_MSS_mss_cfg.xml` into the project's `fpga_design/design_description` folder and delete the default `.xml`, so only one remains.
4. Put your code under `src/application`. Hart 0 (`e51.c`) mainly handles start-up, so use harts 1 to 4 (`u54_1.c` to `u54_4.c`) for your code.
5. Build a configuration that makes a `.hex` for the eNVM, and put it into the `.job` in place of the HSS (Appendix D, last step: `HSS_HEX` or the eNVM client).

The board then runs your program instead of the HSS, so Linux and RTEMS no longer boot. Program `nmc-fpga.job` again to go back.



SoftConsole: the MSS `.xml` sits under `src/boards/<board>/fpga_design/design_description`. Replace the default one with yours.



Application code lives in `src/application`. Hart0 (`e51.c`) handles startup, so use hart1 to hart4.

Links

- SoftConsole: https://www.microchip.com/en-us/products/fpgas-and-plds/fpga-and-soc-design-tools/soc-fpga/softconsole
- Libero licenses (FPGA Software Licenses): https://www.microchipdirect.com/fpga-software-products
- Libero SoC License Installation Guide: https://ww1.microchip.com/downloads/aemDocuments/documents/FPGA/swdocs/libero/Libero_Installation_Licensing_Setup_User_Guide_2025_1.pdf

Appendix F: Disable Ethernet to save power (Linux)

After the image is flashed (Step 5A or 5B), the Ethernet PHYs keep drawing power even with no cable connected. Taking the Linux network interfaces down powers the PHYs down, which dramatically reduced the NMC's power draw in testing. This step is optional. TFTP reflashing is unaffected, because it runs in U-Boot with its own network stack.

A. Find the interface names. U-Boot calls the interfaces `eth0` and `eth1`, but Linux names them `end0` and `end1`. Run this at the Linux prompt on UART1:

```
ip -br link
```

B. Turn Ethernet off (until the next reboot).

```
ip link set end0 down
ip link set end1 down
ip -br link
```

Both interfaces should now show `DOWN`. Watch the supply current to confirm the drop.

C. Turn Ethernet back on.

```
ip link set end0 up
ip link set end1 up
```

Use the UART1 console to run this when Ethernet is off. Allow a few seconds for the link to come up. If you use a static IP address, add it again, or restart your DHCP client if you use DHCP.

D. Keep Ethernet off after a reboot. Create a small systemd service:

```
cat > /etc/systemd/system/eth-down.service << 'EOF'
[Unit]
Description=Bring Ethernet down to save power
After=network.target

[Service]
Type=oneshot
ExecStart=/sbin/ip link set end0 down
ExecStart=/sbin/ip link set end1 down

[Install]
WantedBy=multi-user.target
EOF
systemctl enable eth-down.service
```

Run `which ip` first and use that path in the two `ExecStart` lines if it is not `/sbin/ip`. Reboot, then run `ip -br link` to check that both interfaces come up `DOWN`. If something else in the image brings them back up, it must be told to leave `end0` and `end1` alone.

All steps, including step D, were tested on the NMC with both the Buildroot and the Yocto image.

Notes

1. With Ethernet off, UART1 is the way into the board.
2. Flashing a new image replaces the root filesystem, so the service in step D has to be created again after each reflash. To avoid that, add it to the image (a Buildroot rootfs overlay or a Yocto recipe) so every image build includes it.
3. Reflashing over TFTP still works with Ethernet disabled in Linux.

More information

All of these are in the `nmc-bsp` repository (https://github.com/nextage-llc/nmc-bsp):

- `README.md`: overview, how the board boots, and where to start
- `fpga/README.md`: programming the board and changing the FPGA design
- `nmc-linux-bsp/README.md` and `nmc-rtems-bsp/README.md`: building Linux and RTEMS
- `BOARD_INTERFACES.md`: connector pins and the matching Linux devices
- `CHANGELOG.md`: versions, changes and known issues in each release